Deliverable 3 (Iteration 2 / Sprint 2)

Subject Name: Fundamentals of Software Engineering

Project Name: DataPulse - Enterprise eCommerce Analytics Platform

Group Members and Roll Numbers:

Muhammad Umar Nadeem (24I-2513)

Rafay Mir Khattak (24I-2603)

Samiullah Shahid (24I-2520)

Project Title

DataPulse - Enterprise eCommerce Analytics Platform

A. Sprint Backlog for Sprint 2

A.a Module Selected for Sprint 2

Module Name: Analytics Core and Dashboard Integration Module

A.b User Stories and Sub User Stories

US-01: As an Admin, I want to authenticate into DataPulse so that only authorized users can access analytics.

- US-01.1: Validate user credentials via login form.

- US-01.2: Route successful login to enterprise hub.

- US-01.3: Support logout flow from profile panel.

US-02: As a Data Engineer, I want to ingest raw CSV datasets to cloud tables so that source data is centrally available.

- US-02.1: Read all raw CSV files from data/raw directory.

- US-02.2: Standardize column names for SQL compatibility.

- US-02.3: Upload each dataset to PostgreSQL using chunked writes.

US-03: As a Data Engineer, I want to transform raw tables into a processed master dataset so that dashboard queries are reliable.

- US-03.1: Merge orders, items, products, customers, and payments.

- US-03.2: Convert timestamp fields and handle invalid dates.

- US-03.3: Engineer delivery and delay features.

- US-03.4: Publish processed_sales_data table.

US-04: As a Business Analyst, I want KPI and trend dashboards so that I can monitor revenue and order performance.

- US-04.1: Display revenue, orders, customers, and AOV metrics.

- US-04.2: Support date-range filtering from sidebar.

- US-04.3: Render area, pie, and comparison charts.

- US-04.4: Show master data table for drill-down.

US-05: As a Business Analyst, I want forecast visibility so that I can compare projected and actual sales.

- US-05.1: Load forecast table if available.

- US-05.2: Overlay forecast and actual trend lines.

- US-05.3: Fallback to baseline when forecast is unavailable.

US-06: As an Admin, I want a pipeline monitoring page so that I can check infrastructure readiness.

- US-06.1: Provide pipeline status panel in navigation.

- US-06.2: Trigger database connectivity checks.

- US-06.3: Display diagnostic status messages.

A.c Trello Board Update

Sprint board updated with Backlog, To Do, In Progress, Testing, and Done columns. Cards are linked to US-01 through US-06 and assigned to respective members.

A.d Trello Screenshots

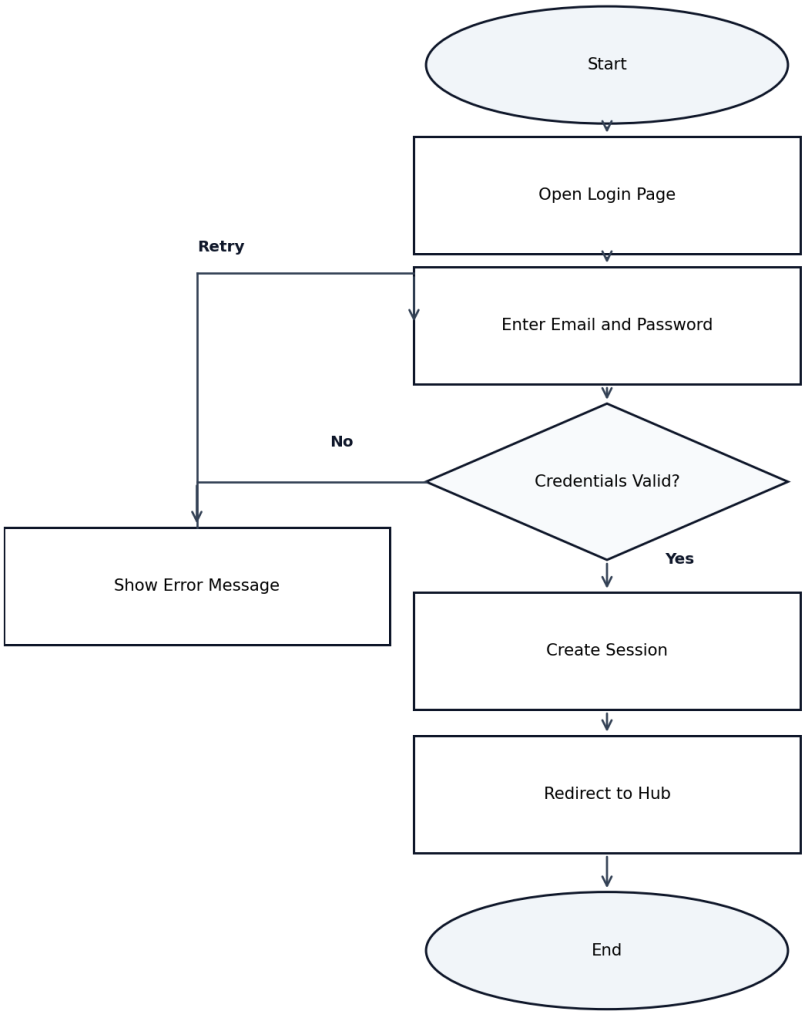
Three Sprint 2 Trello board screenshots are included in the submission package: board overview, in-progress cards, and completed cards.

B. Design

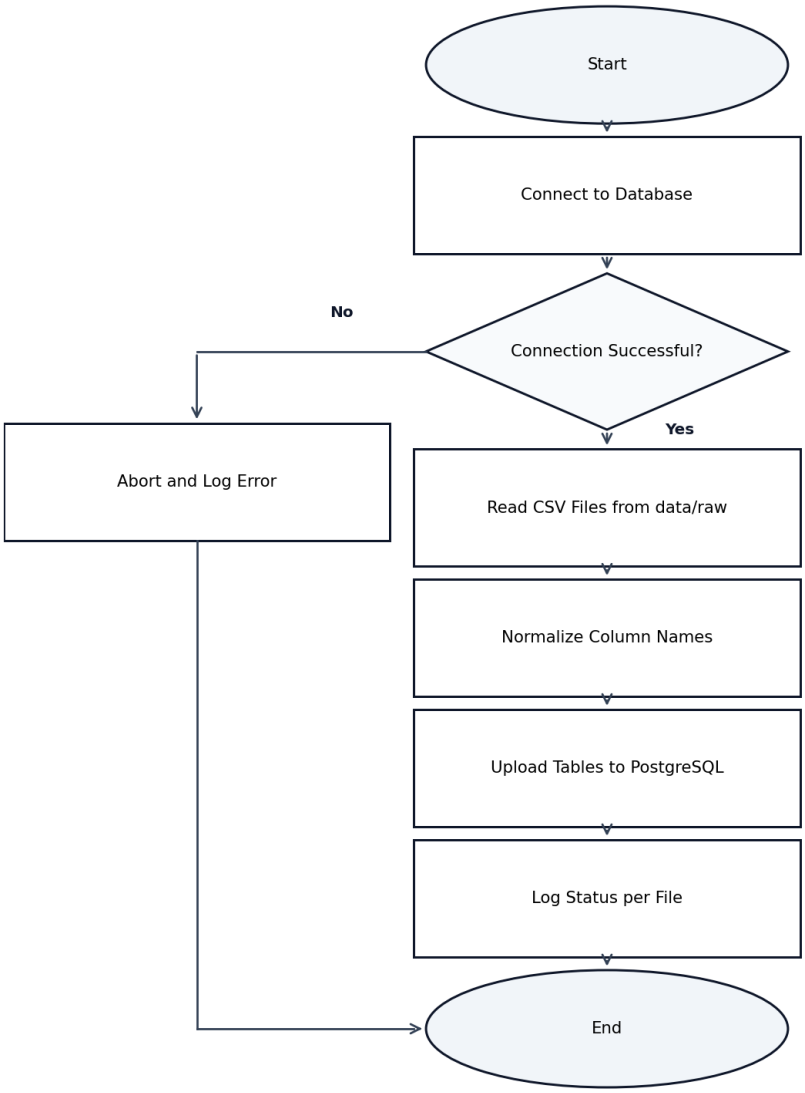
B.a Activity Diagrams for Main Processes

The following six activity diagrams represent the primary workflows in DataPulse.

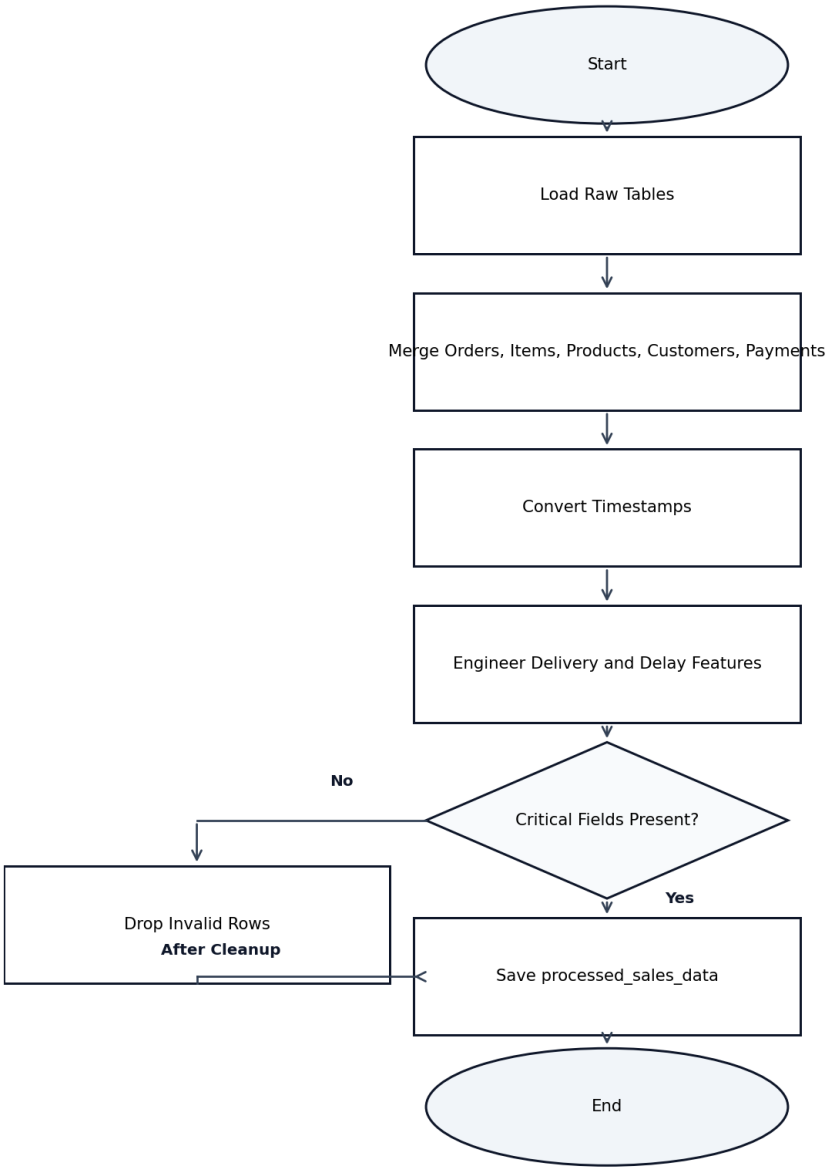
Activity Diagram: User Login



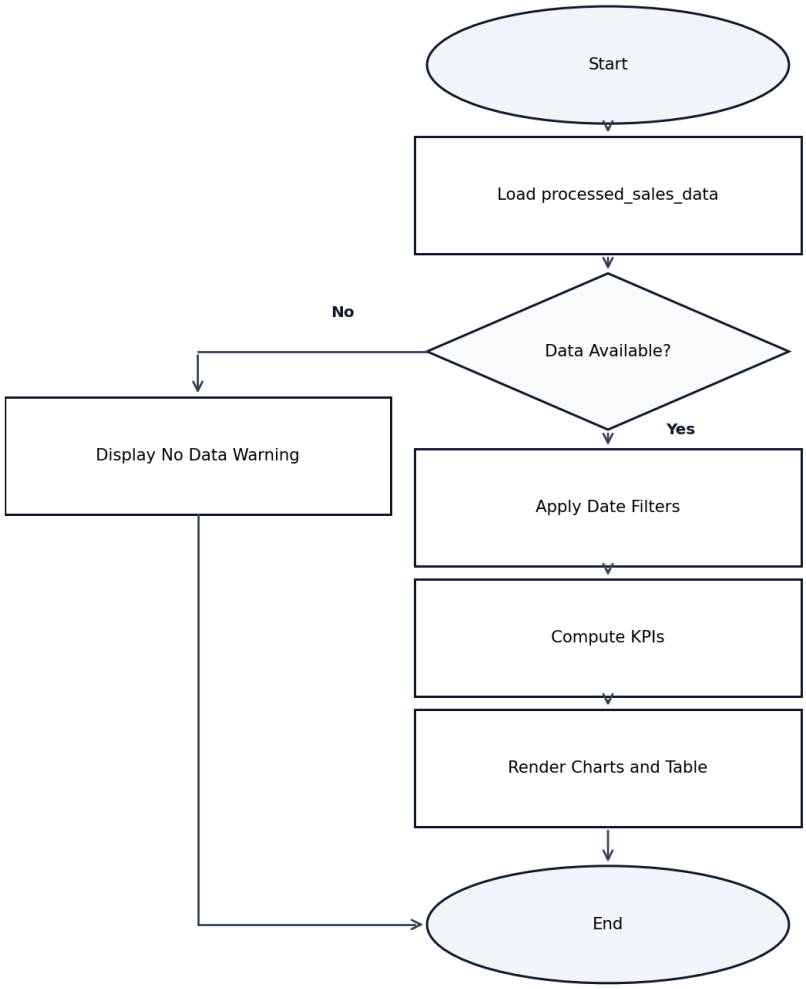
Activity Diagram: Data Ingestion



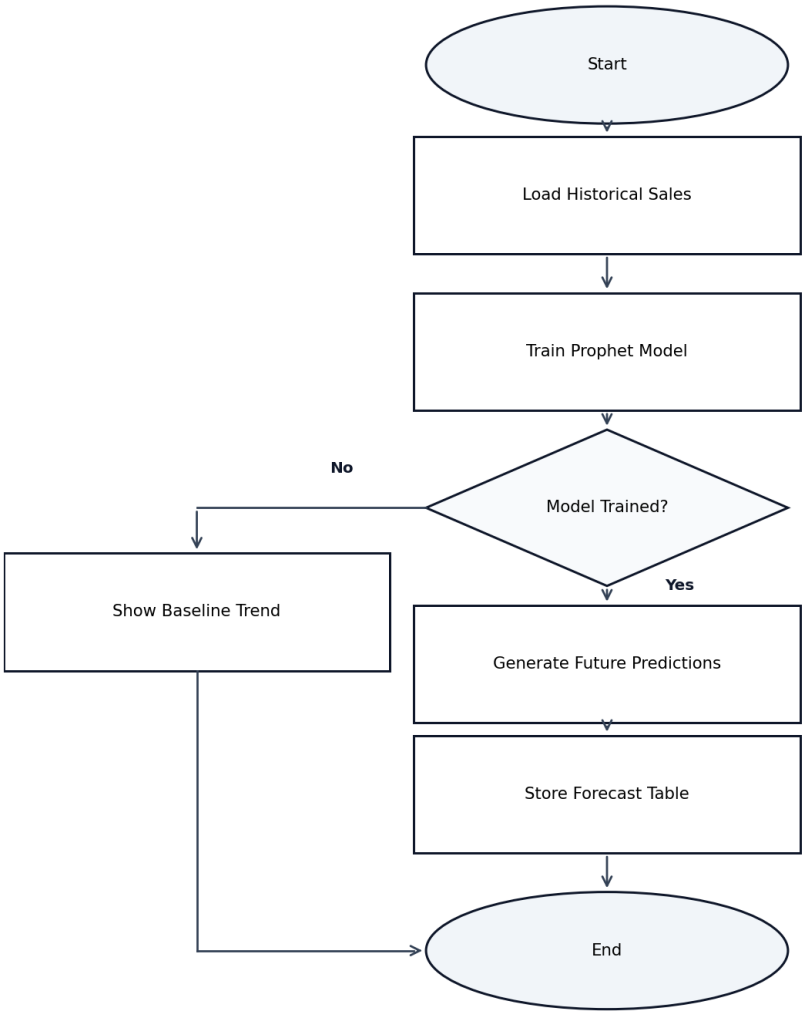
Activity Diagram: Data Transformation



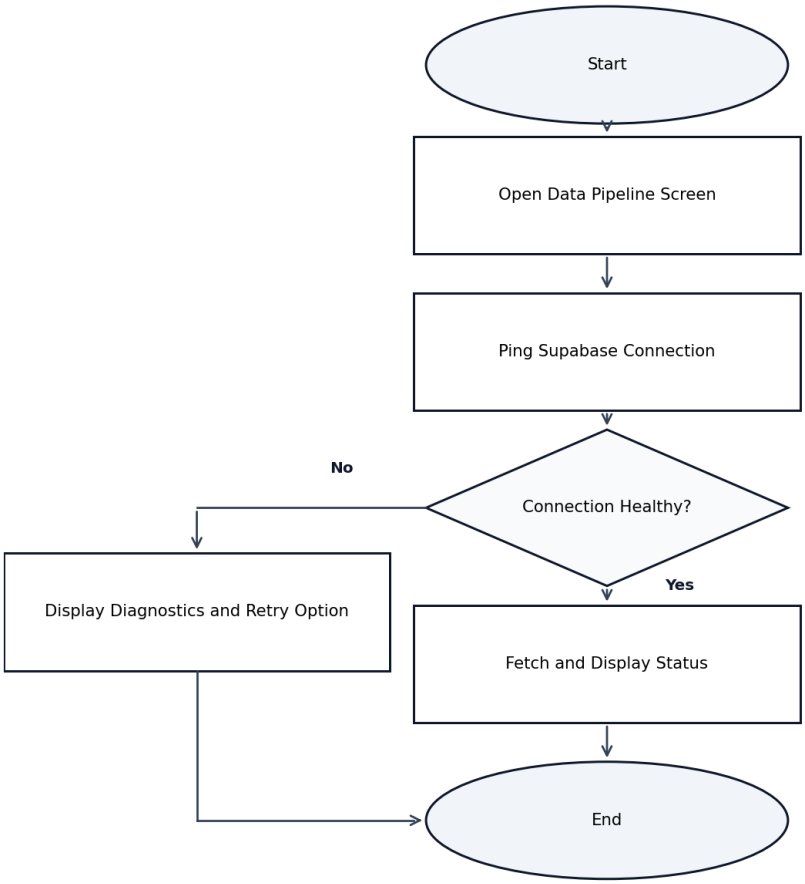
Activity Diagram: Business Insights



Activity Diagram: Sales Forecast

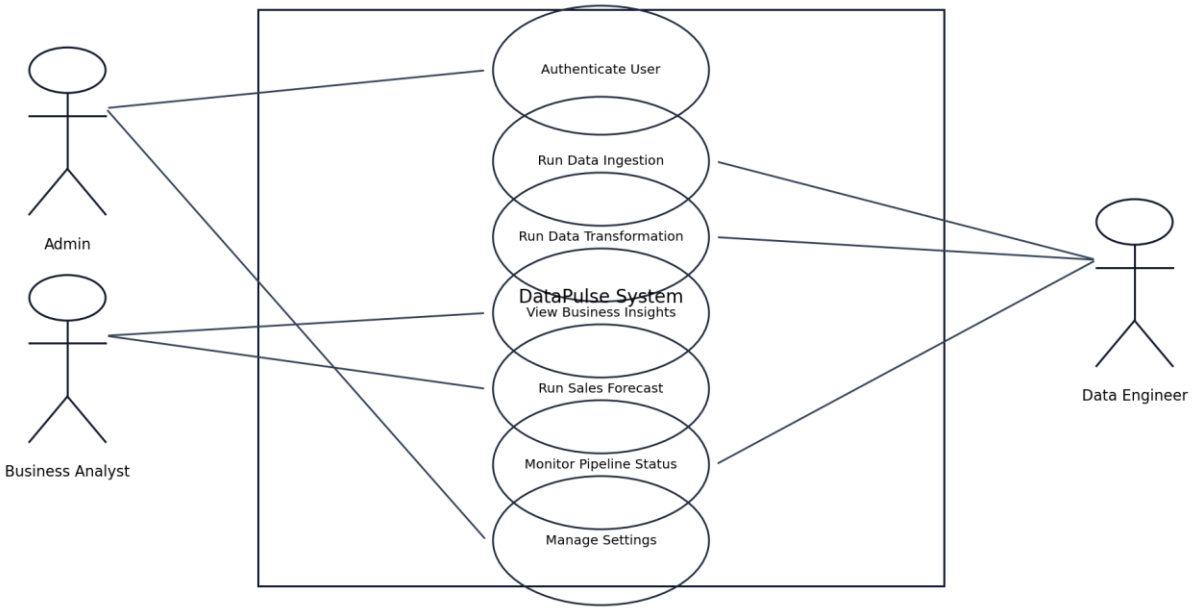


Activity Diagram: Pipeline Monitoring



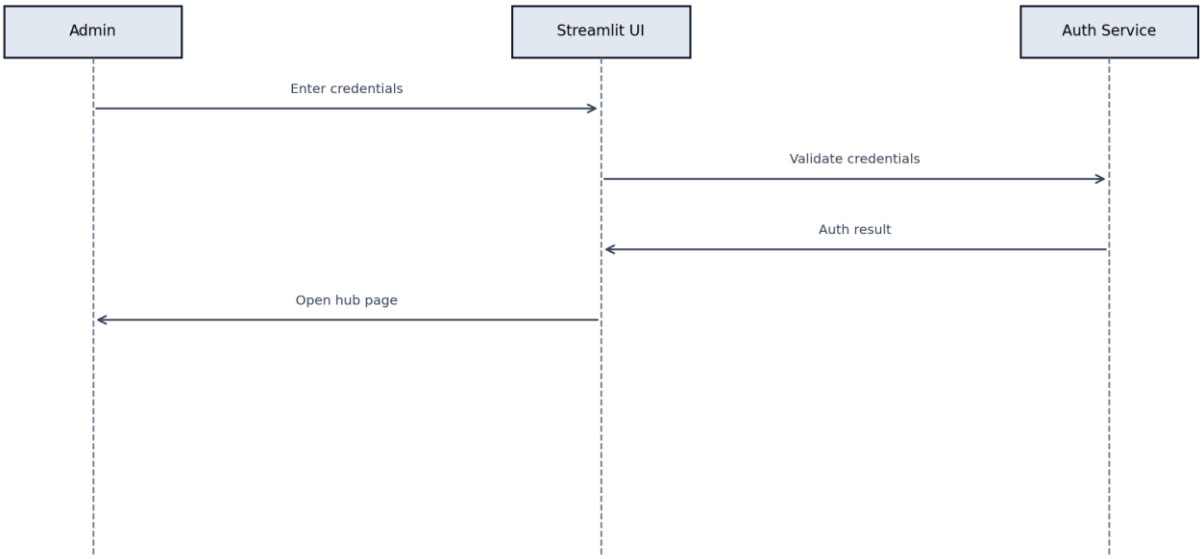
B.b Use Case Diagram

Use Case Diagram: DataPulse

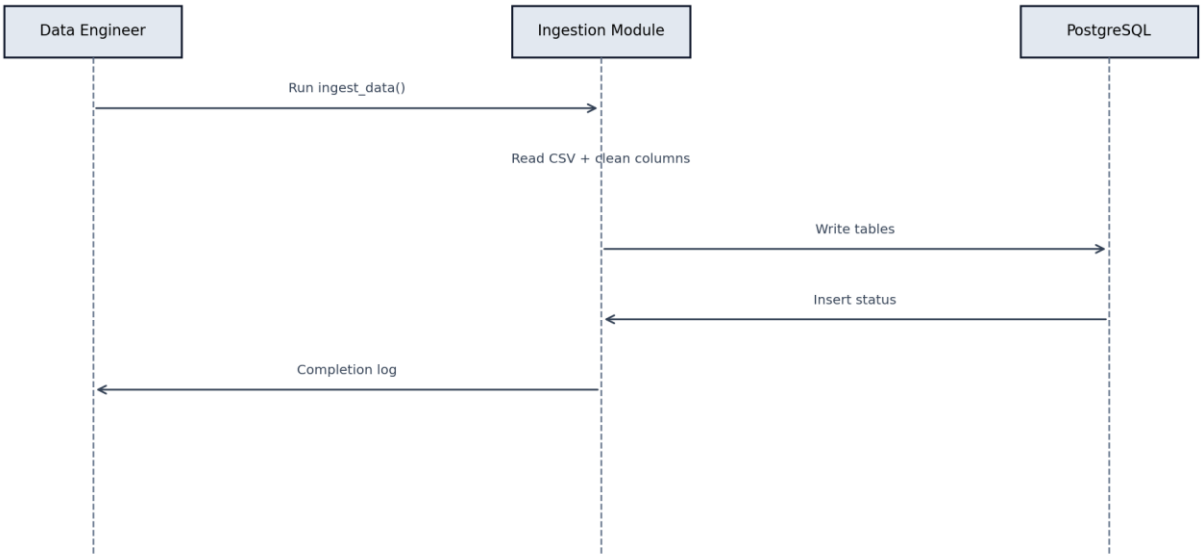


B.c Sequence Diagrams for Main Activities

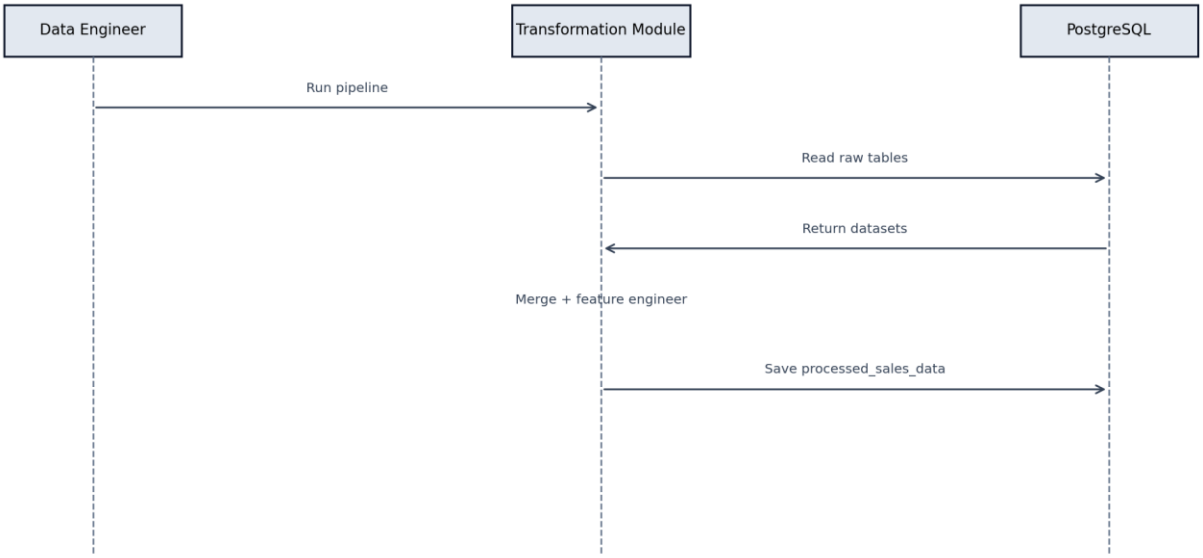
Sequence Diagram: User Login



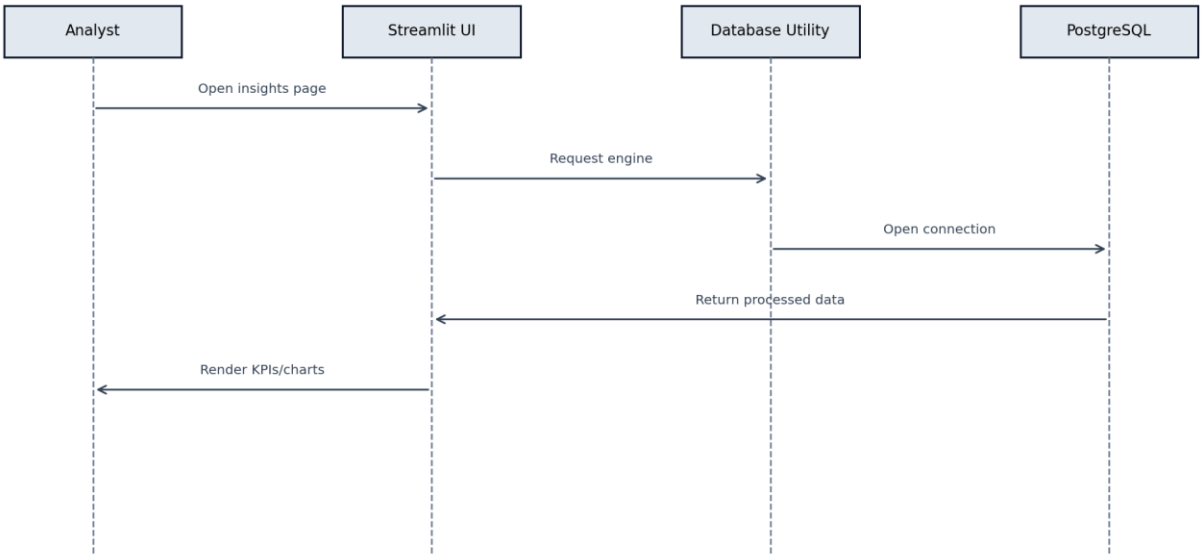
Sequence Diagram: Data Ingestion



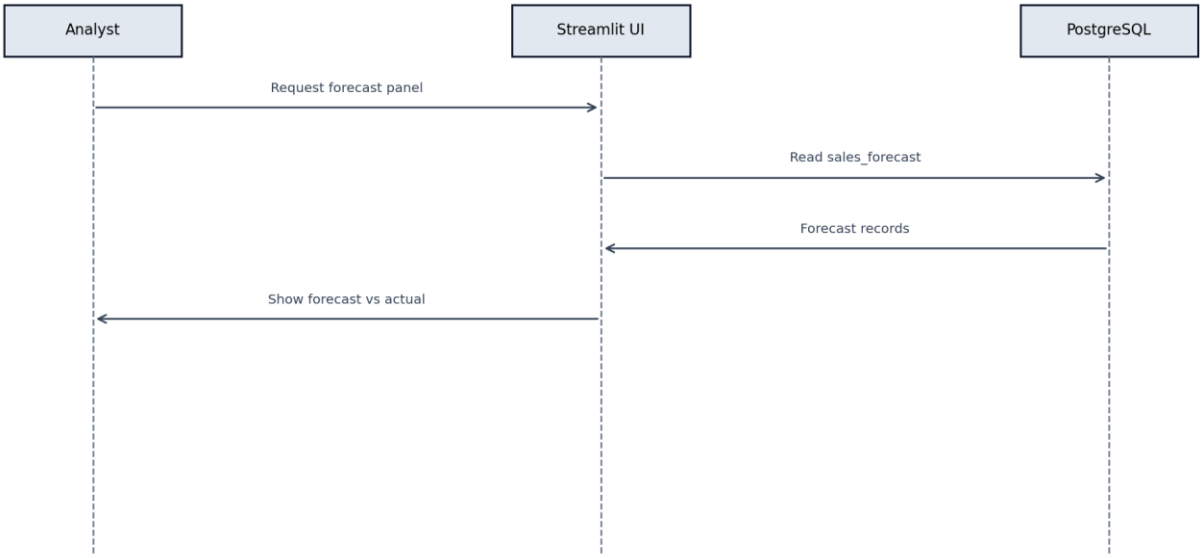
Sequence Diagram: Data Transformation



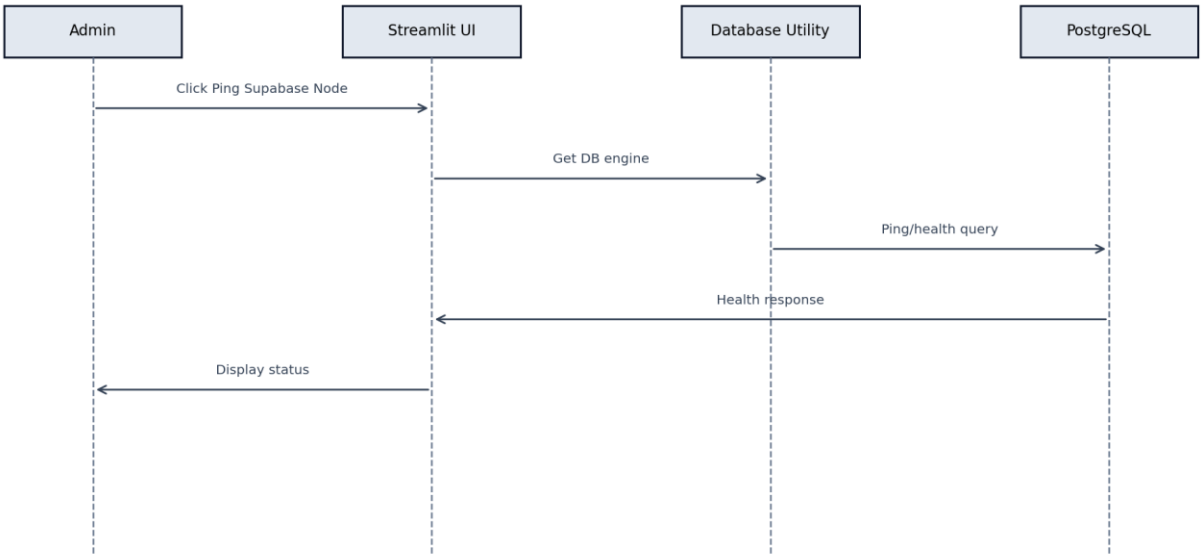
Sequence Diagram: Business Insights Retrieval



Sequence Diagram: Sales Forecast Display

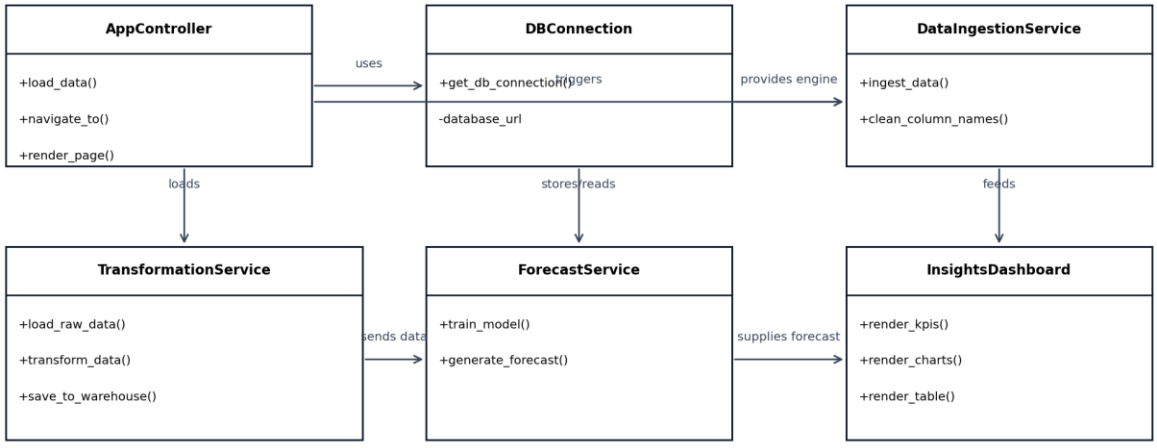


Sequence Diagram: Pipeline Status Check



B.d Class Diagram

Class Diagram: DataPulse



Chapter 1: Introduction

1.1 Project Background

DataPulse is a cloud-enabled analytics platform that transforms raw eCommerce records into actionable business insights.

1.2 Problem Statement

Organizations with growing transaction volumes often lack a unified analytics pipeline. Data remains fragmented across files and tables, delaying decisions and increasing reporting inconsistencies.

1.3 Objectives

- Design a robust raw-to-processed data pipeline for Olist datasets.

- Deliver interactive dashboards for KPI and trend monitoring.

- Provide forecasting support for short-term planning.

- Document architecture, requirements, and UML models for maintainability.

1.4 Scope of the Project

The scope of DataPulse for Iteration 2 covers the end-to-end path from raw transactional data ingestion to interactive business insight delivery, with an emphasis on foundational reliability rather than full production hardening. The project includes four major functional boundaries in this sprint: data ingestion from local CSV datasets, data transformation into a consolidated analytics table, dashboard-driven visualization for business users, and preliminary forecasting integration. Within this scope, the team focuses on workflow continuity, so that each stage feeds the next without manual reformatting or undocumented logic. This means ingestion must produce consistent table schemas, transformation must apply deterministic join and cleaning behavior, and the dashboard must query processed data in a way that reflects current records and selected filters.

The project also includes user-facing navigation and module separation in the Streamlit interface. A login gateway, hub page, business insights screen, pipeline monitor view, and settings page are part of this sprint deliverable. These interfaces are included as implemented components, while UI screenshots and final polish assets are intentionally excluded from this document per team submission preference and will be inserted later by the group lead. On the technical side, Supabase PostgreSQL serves as the persistent cloud datastore, and Python modules provide the service logic. This establishes a scalable baseline architecture suitable for iterative extension in future sprints.

Out of scope for Sprint 2 are advanced enterprise controls such as fine-grained role based access control, production grade observability stacks, comprehensive API layers, and large scale performance optimization under heavy concurrent load. The current implementation

targets validated core flows, not final deployment maturity. Similarly, forecasting currently emphasizes integration readiness and baseline analytics value rather than full model lifecycle management with automated retraining and model governance. Mobile-native clients, multilingual support, and external ERP integrations are deferred to future phases.

Another defined boundary is data source scope. Sprint 2 uses the provided Olist datasets as canonical inputs and does not yet include dynamic ingestion from third-party APIs or streaming brokers. This controlled scope allows deterministic testing and clear traceability from source files to processed outputs. The documentation scope, however, is intentionally broad. It includes requirements, UML designs, sprint planning, module descriptions, testing approach, feasibility assessment, and implementation status mapping. This ensures the report is academically complete even where selected implementation features remain under active development.

Therefore, the Sprint 2 scope can be summarized as a robust analytics MVP: complete enough to demonstrate integrated system behavior, structured enough to support team collaboration, and documented enough to satisfy deliverable standards and enable next sprint acceleration.

1.5 Overview of the System

The system includes ingestion, transformation, and dashboard modules built in Python, with Supabase PostgreSQL as the central data layer and Streamlit as the user-facing application.

Introduction (Minimum 1200 Words)

DataPulse is an enterprise-oriented analytics platform designed to convert fragmented eCommerce records into decision-ready intelligence. The project is centered on the Olist dataset ecosystem and addresses a common problem in digital commerce operations: large volumes of useful data exist, but they are spread across many tables, stored in raw formats, and often disconnected from real business decision cycles. In many organizations, executives and analysts still rely on static exports and manually prepared spreadsheets that become outdated quickly. DataPulse tackles this challenge by introducing a coherent data pipeline, a cloud-backed storage layer, and a modern interactive analytics interface built with Streamlit. The system has been architected so that technical users can perform ingestion and transformation reliably, while non-technical users can access key indicators without writing code. This balance between technical rigor and business usability is a core design principle of the project.

At its foundation, DataPulse follows a practical extract-load-transform sequence with explicit module boundaries. The ingestion component reads raw CSV files from the project repository, standardizes naming conventions, and uploads records into Supabase PostgreSQL tables. This stage is intentionally transparent and deterministic. Every file is mapped to a known table, and consistent normalization rules are applied so that later transformations do not fail because of column naming inconsistencies. After ingestion, the

transformation pipeline performs domain-specific joins across orders, items, products, payments, and customer attributes. It then applies date parsing, data quality cleaning, and feature engineering steps such as delivery time and delay calculations. The result is a processed master table that consolidates transaction-level events into analytics-friendly structure. This layer is what powers dashboard queries and forecasting.

The front-end experience is implemented in Streamlit and organized as a module-driven interface. Users authenticate, navigate through an enterprise hub, and access dedicated screens for business insights, data pipeline status, and system settings. The dashboard provides KPIs, category and channel distributions, trend visualizations, and a filtered master data grid. Because dashboards can become misleading if they are not tied to current data, DataPulse is designed with cache-aware loading and direct database reads so that business users obtain timely summaries. The interface emphasizes clarity and operational relevance rather than decorative complexity. In particular, key widgets are tied to workflow questions: total revenue, order volume, customer count, order value, and trend behavior over selected periods.

A major objective for this sprint is to elevate the project from implementation-only status into a documented software engineering artifact. That means technical decisions must be explained, workflows visualized, and development practices tracked through agile outputs. The report therefore includes sprint backlog details, user stories, UML models, module descriptions, and testing structure. By doing so, DataPulse becomes easier to evaluate academically and easier to maintain in team settings. UML deliverables are not treated as isolated diagrams but as design instruments that connect requirements to code. The use case diagram captures external actors and their goals. Activity diagrams decompose main workflows into ordered actions. Sequence diagrams capture interactions between UI, services, and data stores. The class diagram provides a conceptual object model for future expansion, even though the current implementation uses function-centric modules.

From an engineering standpoint, technology choices were made to maximize delivery speed while keeping industrial relevance. Python was chosen because it offers mature libraries for ingestion, transformation, visualization, and machine learning in one ecosystem. Pandas enables efficient manipulation of tabular structures. SQLAlchemy provides controlled connectivity to PostgreSQL. Streamlit drastically reduces front-end overhead and allows rapid production of analyst-facing interfaces. Supabase is used as a managed PostgreSQL backend with cloud accessibility, which simplifies collaboration and deployment for student teams. Plotly delivers interactive graphing and supports business storytelling with minimal code. These choices create a full-stack analytics path without introducing unnecessary architectural friction for a sprint-based academic project.

The business value of DataPulse lies in shortening the loop between raw transactions and actionable decisions. Revenue leaders can examine trend direction, channel behavior, and order performance quickly. Operations teams can inspect delivery and delay indicators

produced in transformation. Data engineers can validate ingestion and monitor whether data is reaching the warehouse as expected. As the forecasting module matures, planning teams can compare predicted sales trajectories against actual performance and adjust inventory, promotions, and staffing plans accordingly. This aligns the project with practical digital commerce needs where timing and clarity often matter as much as model complexity.

The project also addresses quality concerns that emerge in early data products. One common issue is that dashboards are built before data definitions are stabilized. DataPulse avoids this by explicitly engineering the processed_sales_data table as a semantic layer. Another issue is hidden assumptions in joins and date treatment; these are surfaced in the transformation module and documented in the report. A third issue is weak reproducibility. This sprint improves reproducibility by providing scripted diagram generation and document assembly, making deliverables less dependent on manual editing and reducing risk of inconsistency at submission time.

Stakeholder alignment has been a guiding concern in this sprint. Administrators need governance and access control pathways, business analysts need trusted insights, and engineers need operable pipelines. The current version covers these expectations at foundational level. The login and navigation workflow structures user entry and access. The business insights page provides decision-focused summaries with filters. The pipeline page supports infrastructure confidence checks. The settings area lays groundwork for configurable behavior. Scrum planning artifacts map these capabilities to user stories so that each increment has traceable intent and measurable completion criteria.

A notable design decision in DataPulse is the preference for explicit data contracts over implicit assumptions. In practical terms, this means each stage of the pipeline expects specific fields and creates explicit outputs that are documented in the report and reflected in UML diagrams. This approach reduces ambiguity when multiple team members work on different modules. For example, if one developer updates ingestion behavior, the transformation contract remains visible and can be validated immediately. In sprint-oriented development, these contracts are essential because rapid iteration often introduces hidden mismatches between source data and interface expectations. By committing to explicit field naming and deterministic joins, the team improved both reliability and debugging speed.

Another key perspective is maintainability under academic project constraints. Student teams frequently experience context switches, limited synchronized working hours, and staggered contribution patterns. A maintainable codebase therefore needs clear structure more than advanced abstraction. DataPulse reflects this by using straightforward module boundaries and verbose, readable transformations. While this may appear less compact than heavily abstracted production code, it is far more effective for collaborative learning and report traceability. Every major business capability in the application maps to a

concrete file or function sequence, which improves review quality and reduces onboarding friction for new contributors.

The system also incorporates graceful degradation principles, especially in analytics views. In real-world analytics products, temporary failures are common: forecast tables may not exist yet, source files may be delayed, or optional fields may be partially missing. Instead of failing hard, DataPulse aims to display fallback states and informative guidance, such as baseline trend views when machine learning output is unavailable. This keeps the dashboard operational and maintains stakeholder trust. Graceful degradation is especially important in sprint demonstrations, where evaluators need to see stable behavior even when some advanced features are still under active development.

Data governance and quality considerations are also central to the platform vision. Even at MVP stage, governance begins with consistent naming conventions, explicit processing logs, and clear ownership of module responsibilities. Quality starts with validation checkpoints, such as null handling in critical timestamps and controlled drops for invalid records. These actions may seem small individually, but together they define whether analytics outcomes are credible. This sprint therefore treats governance as an engineering habit rather than a late-stage compliance layer. As the project evolves, these foundations can support stronger controls such as data lineage tracking, role-based permissions, and audit-ready reporting behavior.

The long-term roadmap for DataPulse includes richer role management, stronger forecasting diagnostics, anomaly detection, and automated scheduling for ingestion and transformation jobs. Additional improvements will likely include model retraining cycles, alerting for data freshness, and expanded export/reporting channels. However, Sprint 2 is intentionally scoped to produce a solid 65 to 70 percent implementation baseline with high-quality design documentation. This aligns with deliverable expectations and creates a stable platform for future iteration.

In conclusion, DataPulse represents a meaningful blend of software engineering discipline and analytics product thinking. The system is not only functional but also increasingly well-structured in terms of requirements traceability, architectural consistency, and documentation completeness. This report captures that evolution and provides a clear view of what has been built, why it was built that way, and how the team will continue development in subsequent sprints.

Chapter 2: System Analysis and Requirements

2.1 Existing System

Before DataPulse, analysis was largely manual and spreadsheet based, with no unified flow from ingestion to visual insight.

2.2 Proposed System

DataPulse introduces an integrated analytics pipeline with cloud persistence, transformation logic, and interactive decision dashboards.

2.3 Stakeholders Identification

- Admin: controls system access and operational checks.

- Data Engineer: runs ingestion and transformation pipelines.

- Business Analyst: consumes KPIs, trends, and forecasts.

- Project Team: develops, tests, and documents system evolution.

2.4 Functional Requirements

- The system shall authenticate users before exposing dashboard modules.

- The system shall read CSV files from the raw data folder and upload them to PostgreSQL.

- The system shall normalize column names before SQL insertion.

- The system shall merge multiple domain tables into a processed sales dataset.

- The system shall compute delivery-related derived fields.

- The system shall display KPI cards and visual analytics for filtered periods.

- The system shall show sales forecast when forecast data is available.

- The system shall provide a pipeline monitoring interface.

2.5 Non-Functional Requirements

- Usability: screens should be readable and navigable for non-technical users.

- Performance: dashboard interactions should remain responsive for expected dataset sizes.

- Reliability: ingestion and transformation should handle missing values gracefully.

- Maintainability: modules should be separated by responsibility and clearly documented.

- Scalability: architecture should support extension to larger datasets and additional modules.

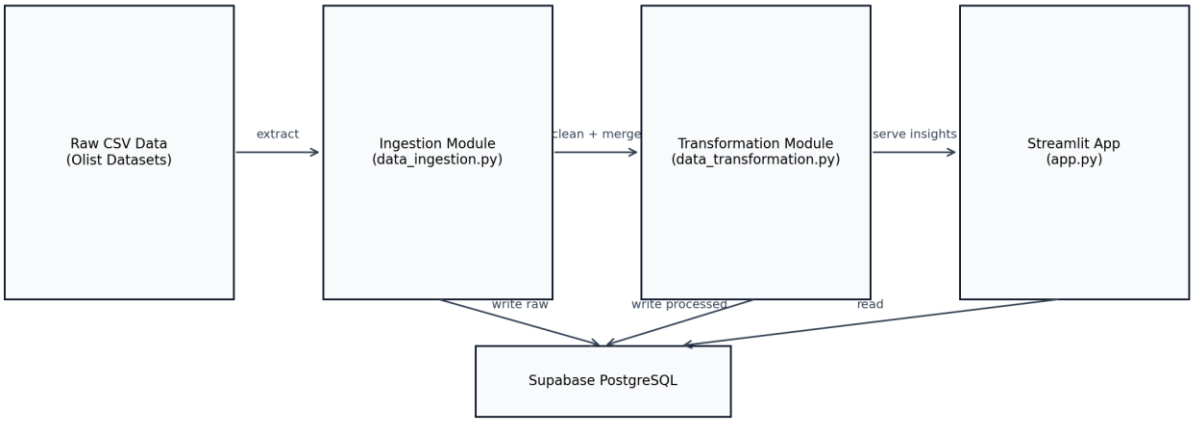
2.6 Feasibility Analysis (Technical, Economic, Operational)

Technical feasibility is high due to mature Python ecosystem and proven cloud database integration. Economic feasibility is favorable because the stack uses open source tools and student-friendly cloud tiers. Operational feasibility is strong as the workflows align with existing team skills and sprint-based development.

Chapter 3: System Design (UML Diagrams)

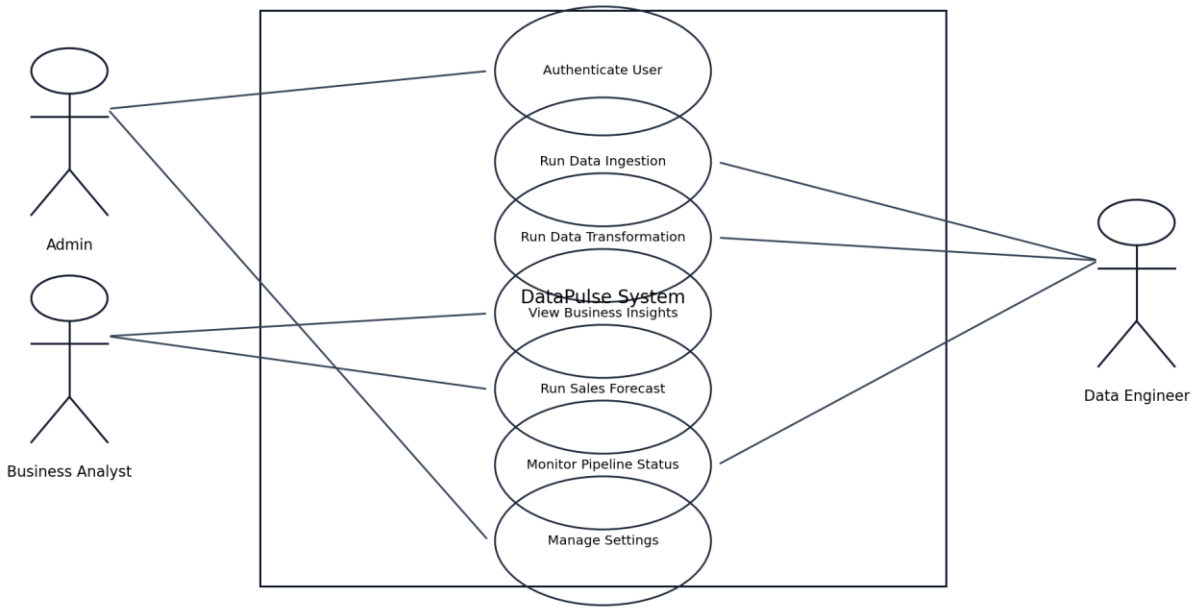
3.1 System Architecture

System Architecture: DataPulse



3.2 Use Case Diagram

Use Case Diagram: DataPulse



3.3 Use Case Descriptions

- Authenticate User: Admin logs in and receives access to authorized modules.

- Run Data Ingestion: Engineer uploads raw files into cloud tables.

- Run Data Transformation: Engineer builds processed analytics dataset.

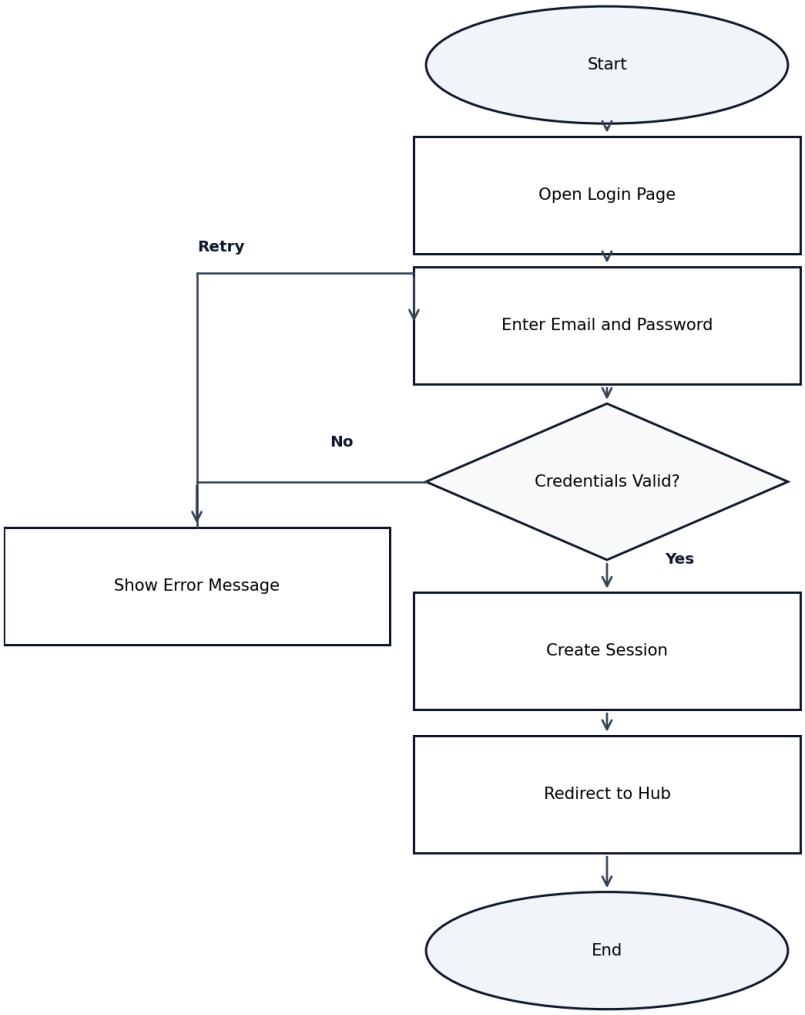
- View Business Insights: Analyst explores KPIs, charts, and data table.

- Run Sales Forecast: Analyst compares projected and actual trends.

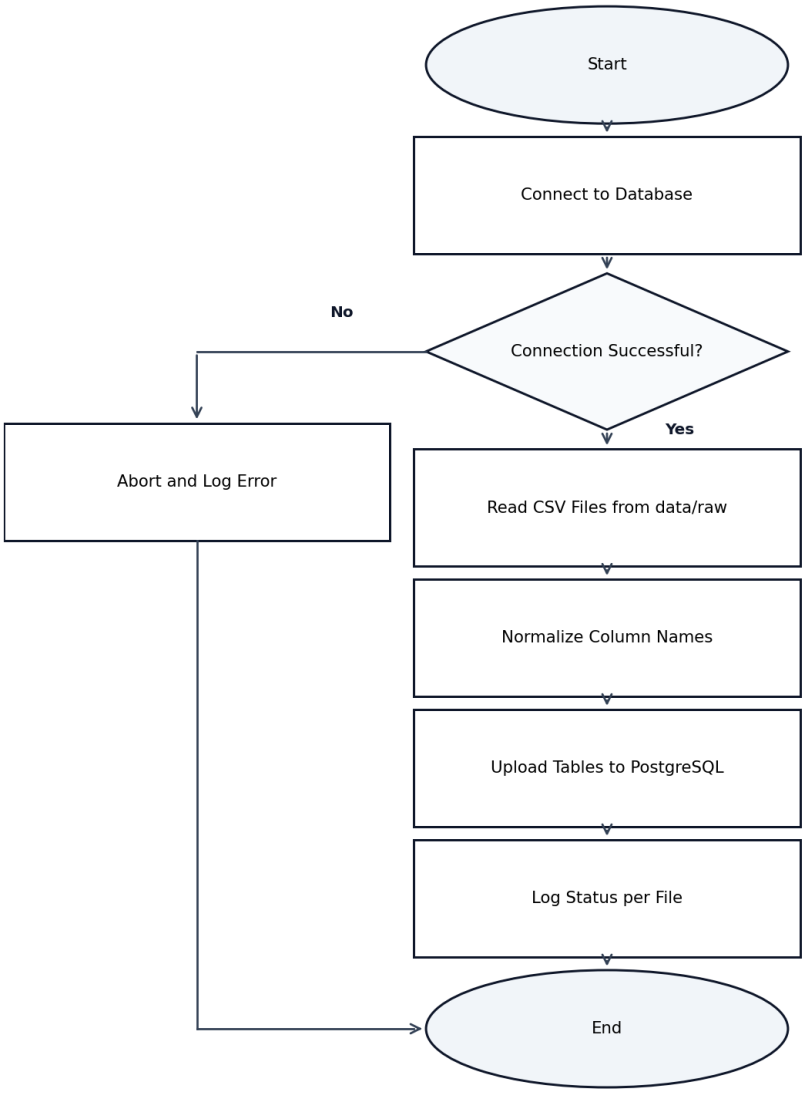
- Monitor Pipeline Status: Admin checks connection and processing state.

3.4 Activity Diagrams

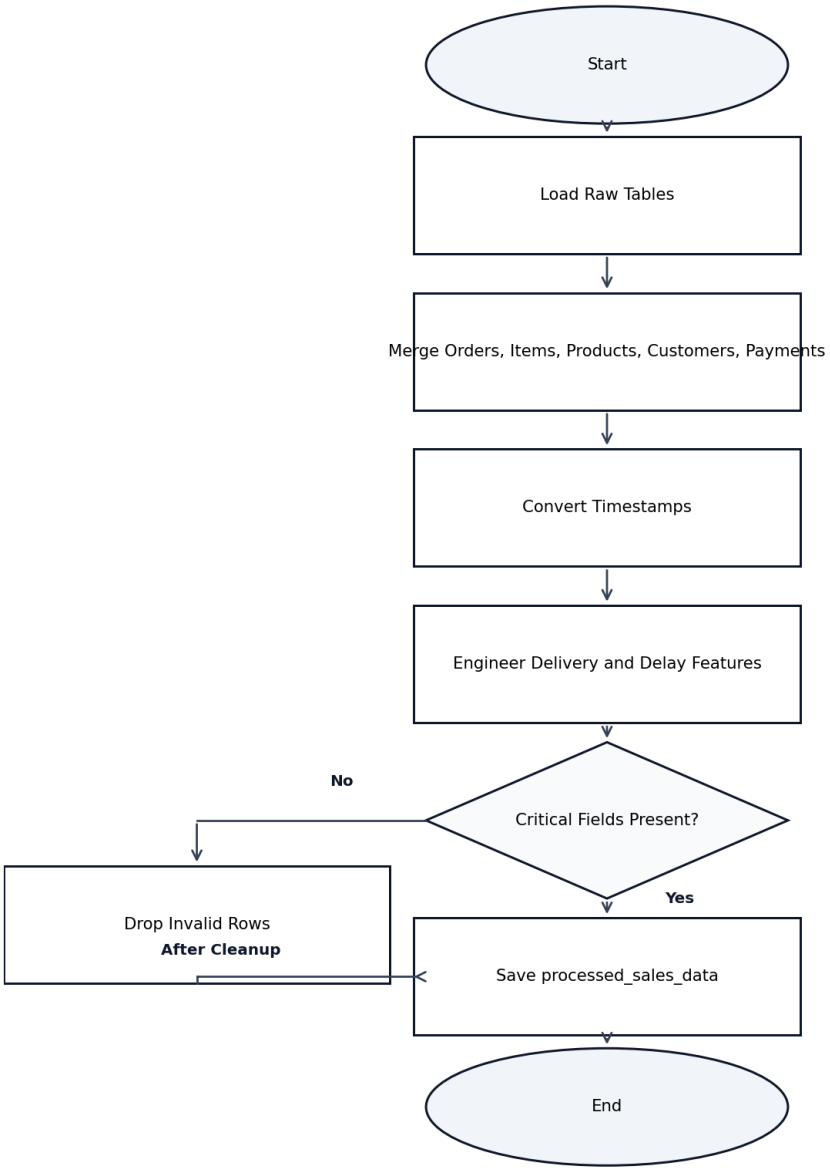
Activity Diagram: User Login



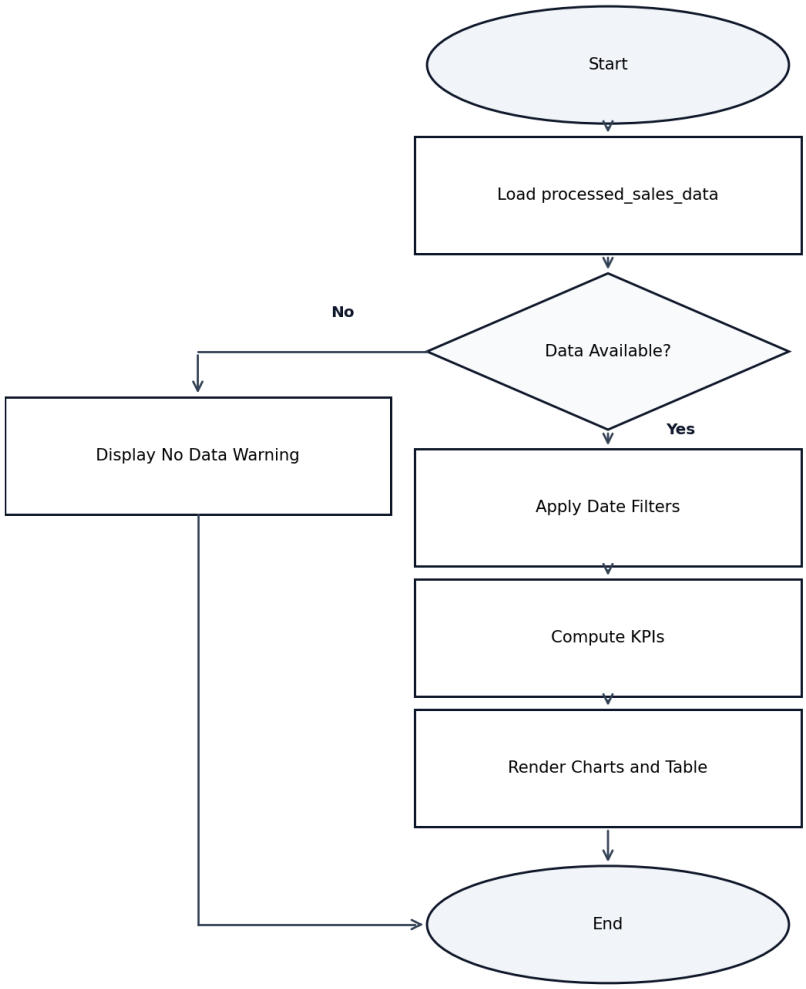
Activity Diagram: Data Ingestion



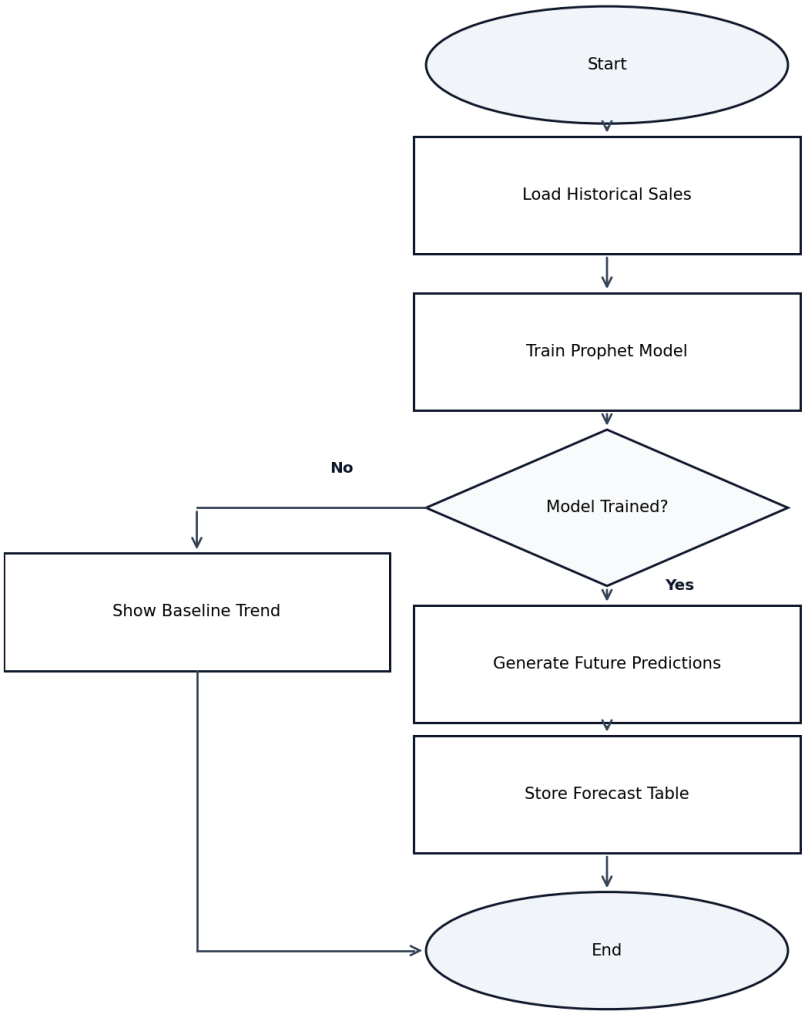
Activity Diagram: Data Transformation



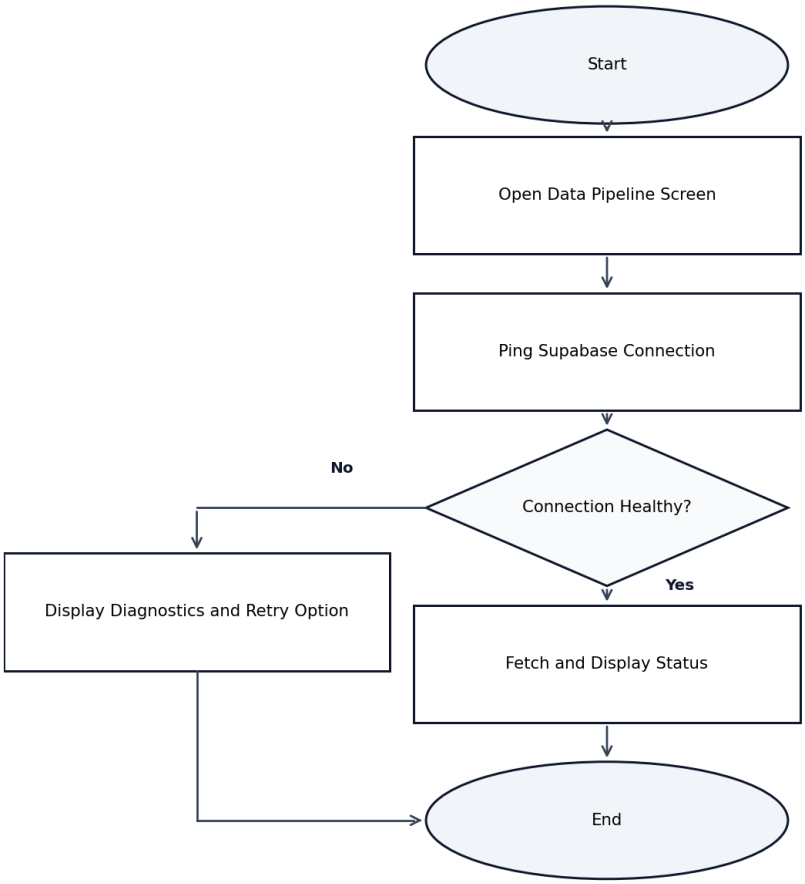
Activity Diagram: Business Insights



Activity Diagram: Sales Forecast

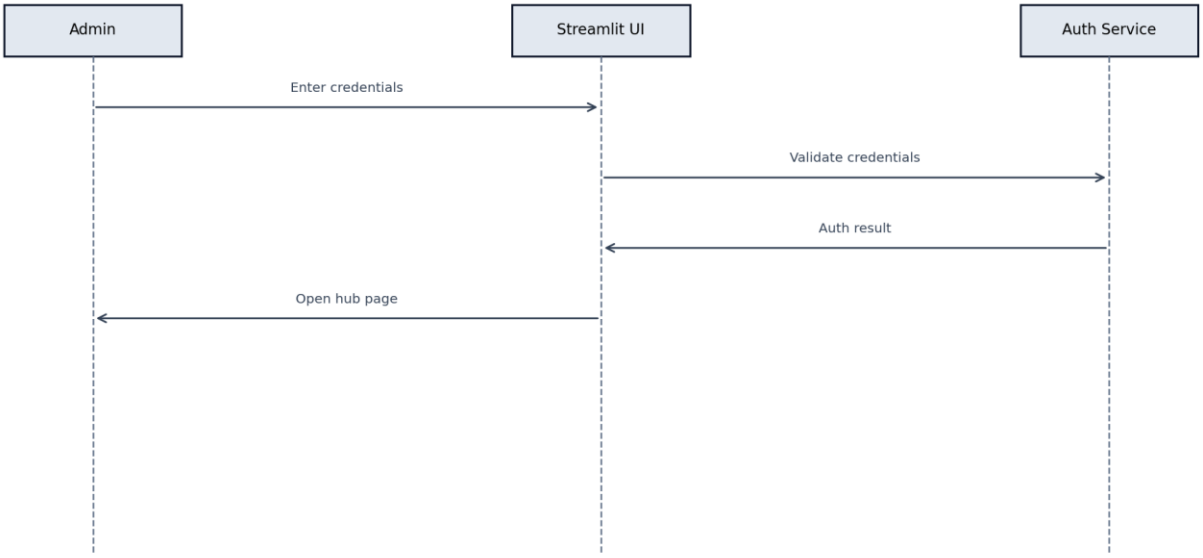


Activity Diagram: Pipeline Monitoring

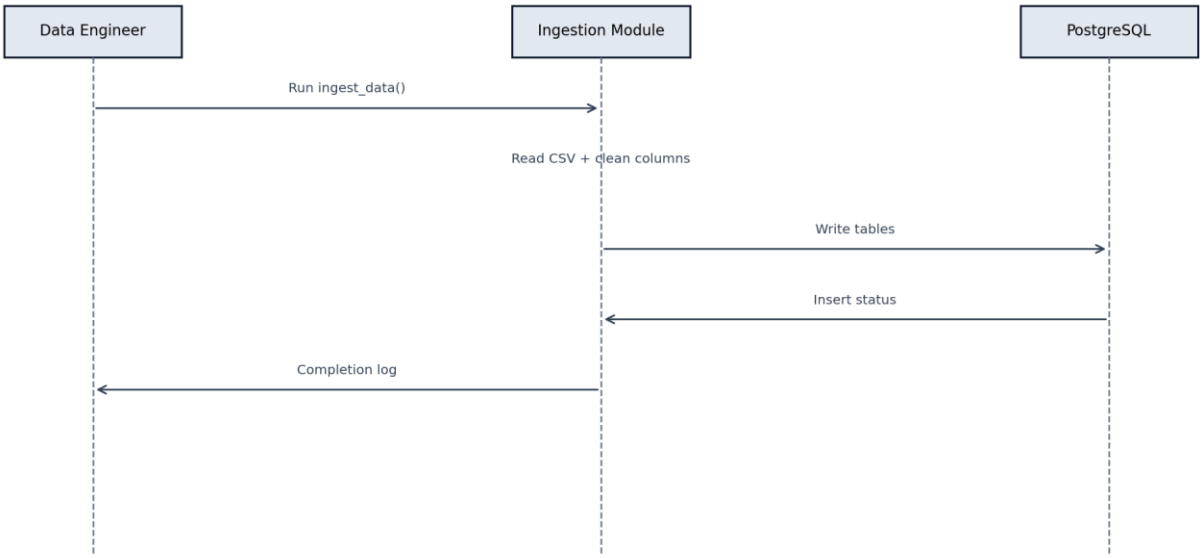


3.5 Sequence Diagrams

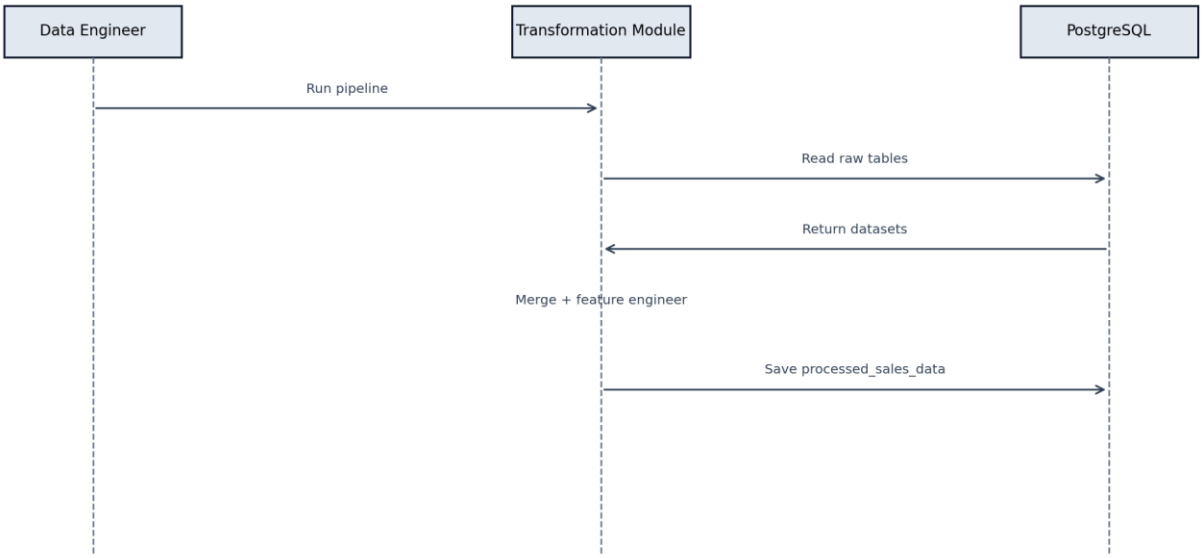
Sequence Diagram: User Login



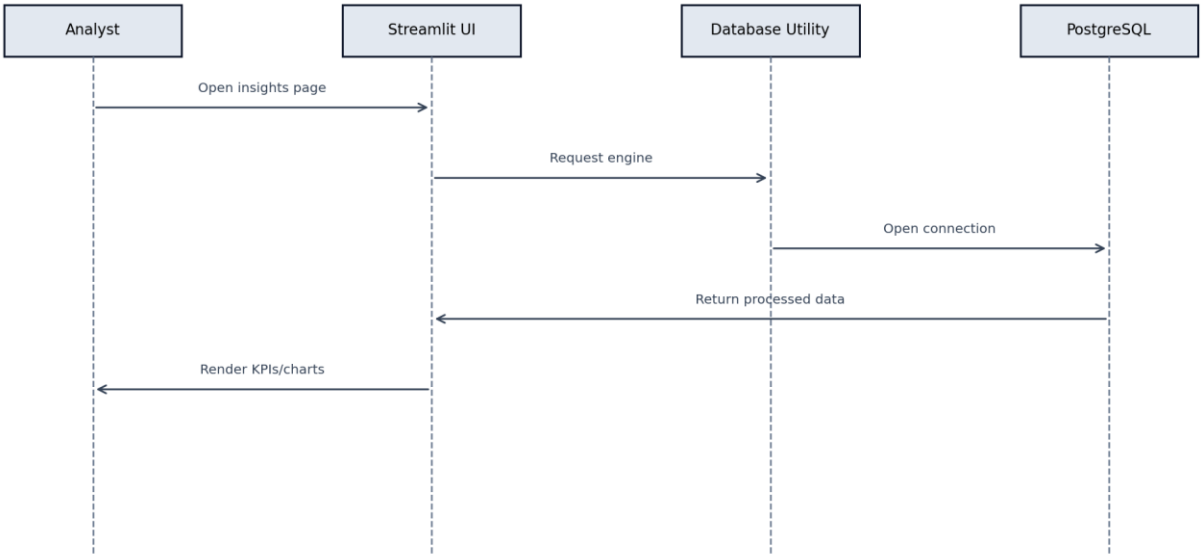
Sequence Diagram: Data Ingestion



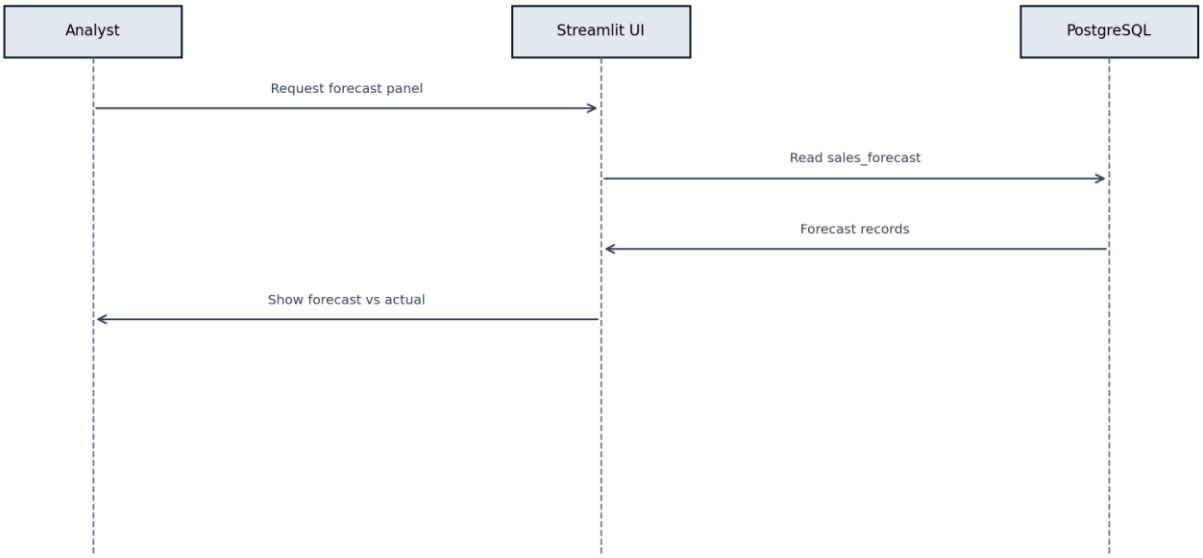
Sequence Diagram: Data Transformation



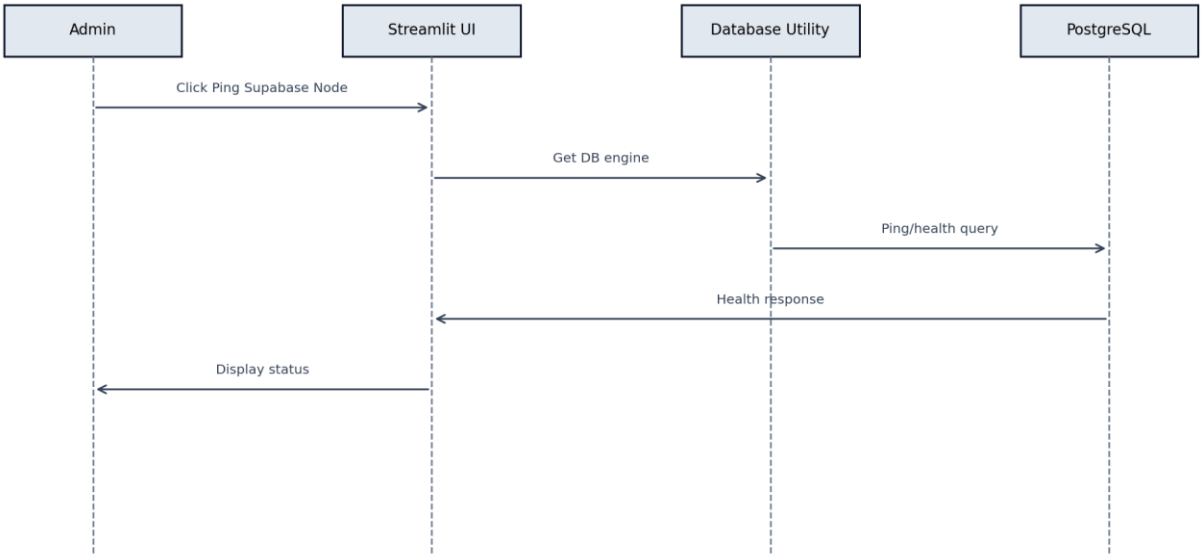
Sequence Diagram: Business Insights Retrieval



Sequence Diagram: Sales Forecast Display

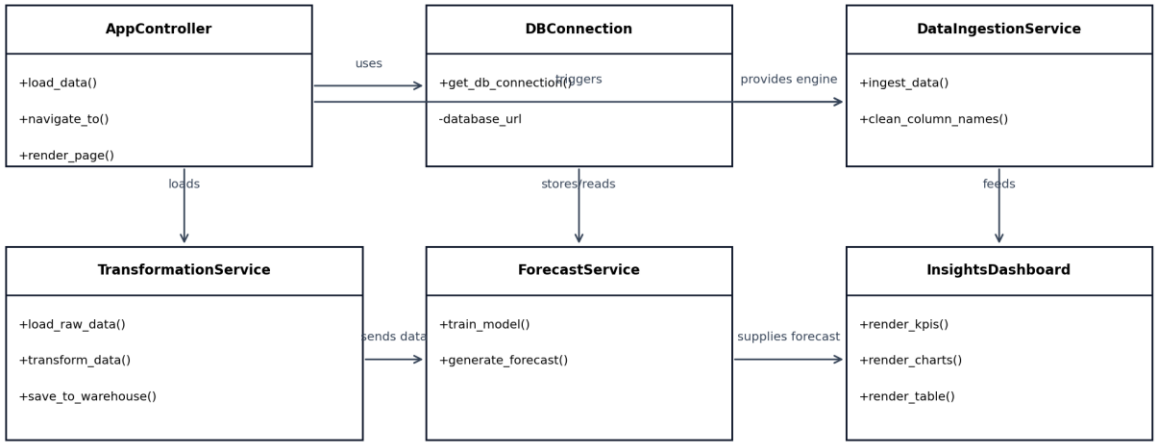


Sequence Diagram: Pipeline Status Check

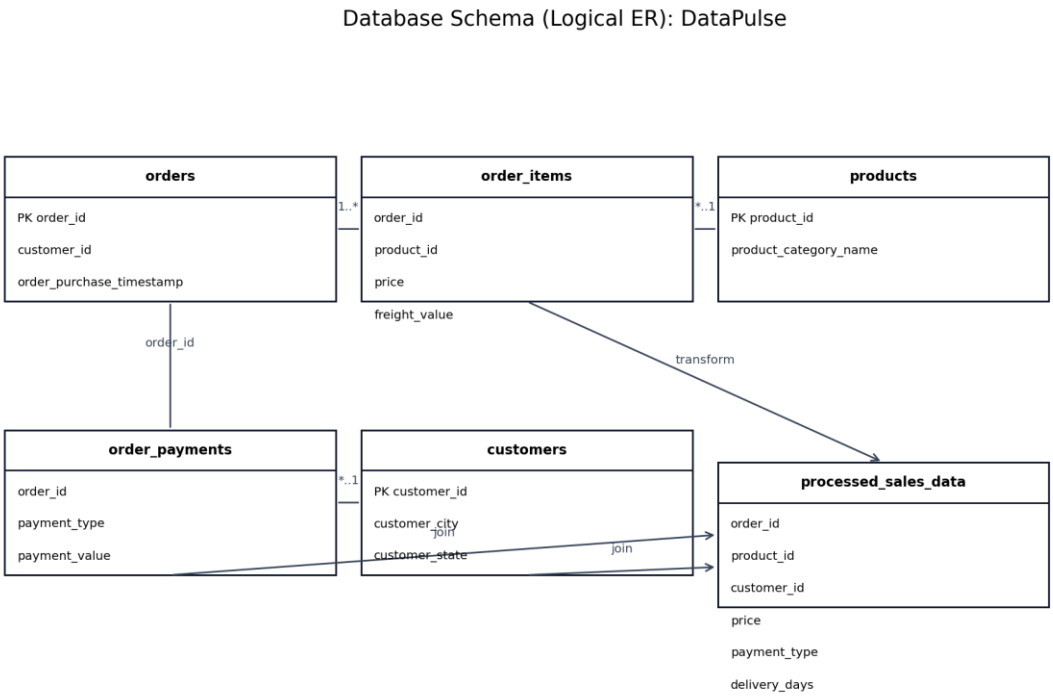


3.6 Class Diagram

Class Diagram: DataPulse



3.7 Database Design (ER Diagram / Schema)



3.8 Interface Design (Wireframes / Prototypes)

UI screens imported from Iteration 1 deliverable for continuity.

Welcome Back

Log in to your DataPulse Enterprise account

Work Email

admin@company.com

Password

••••••••••••

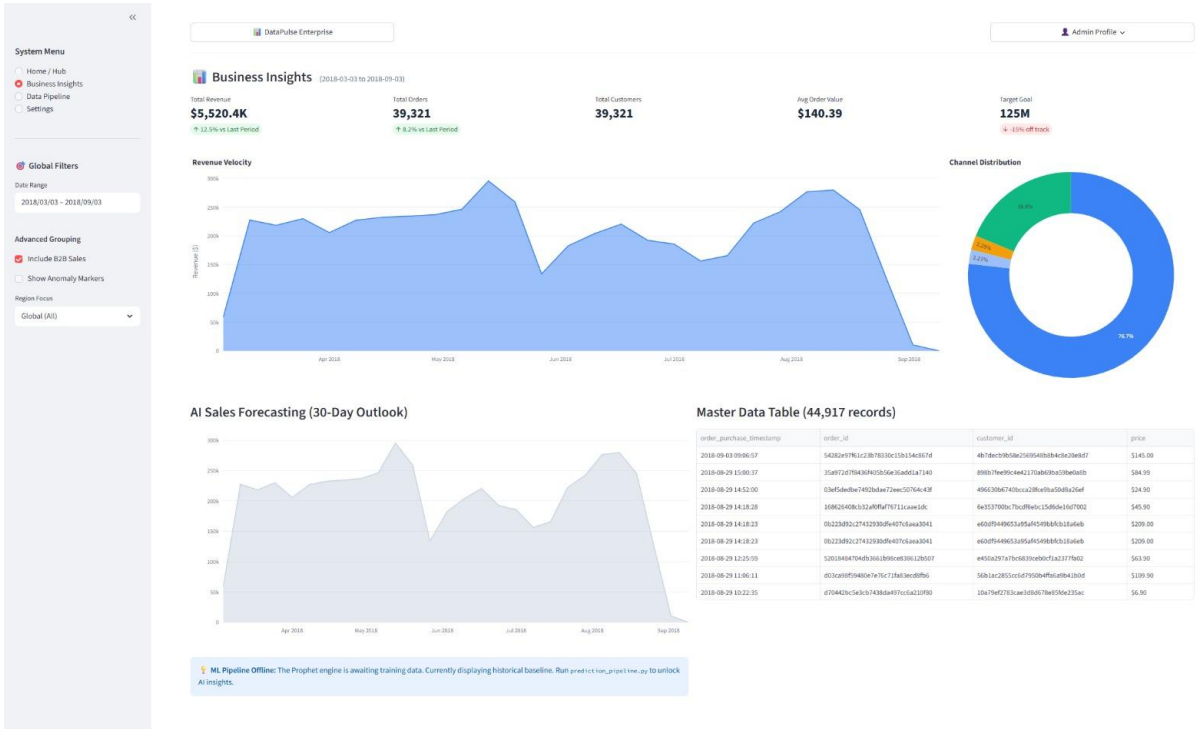
Forgot password?

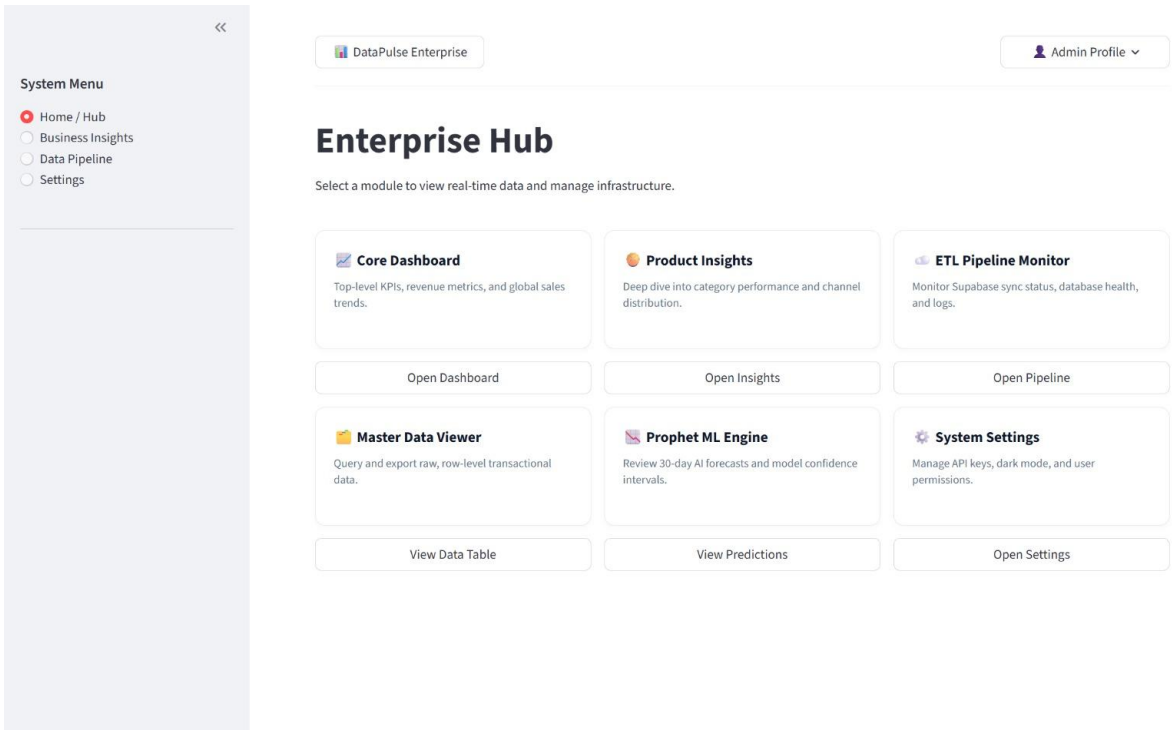
Log In

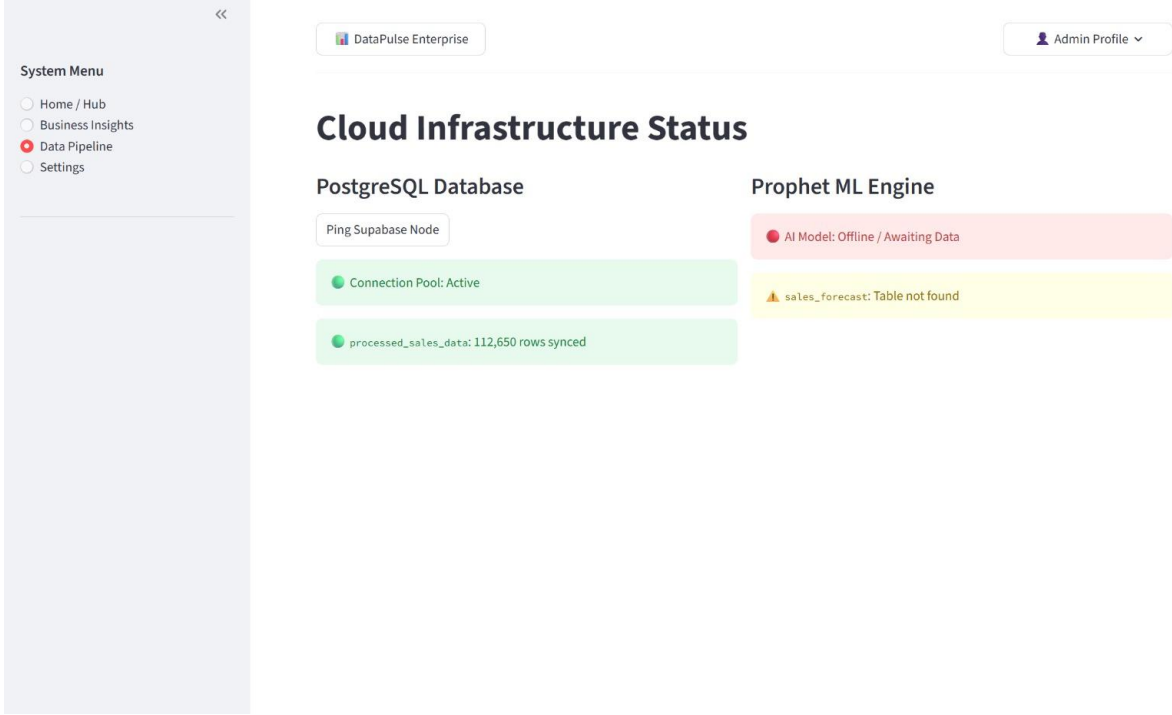
— OR CONTINUE WITH —

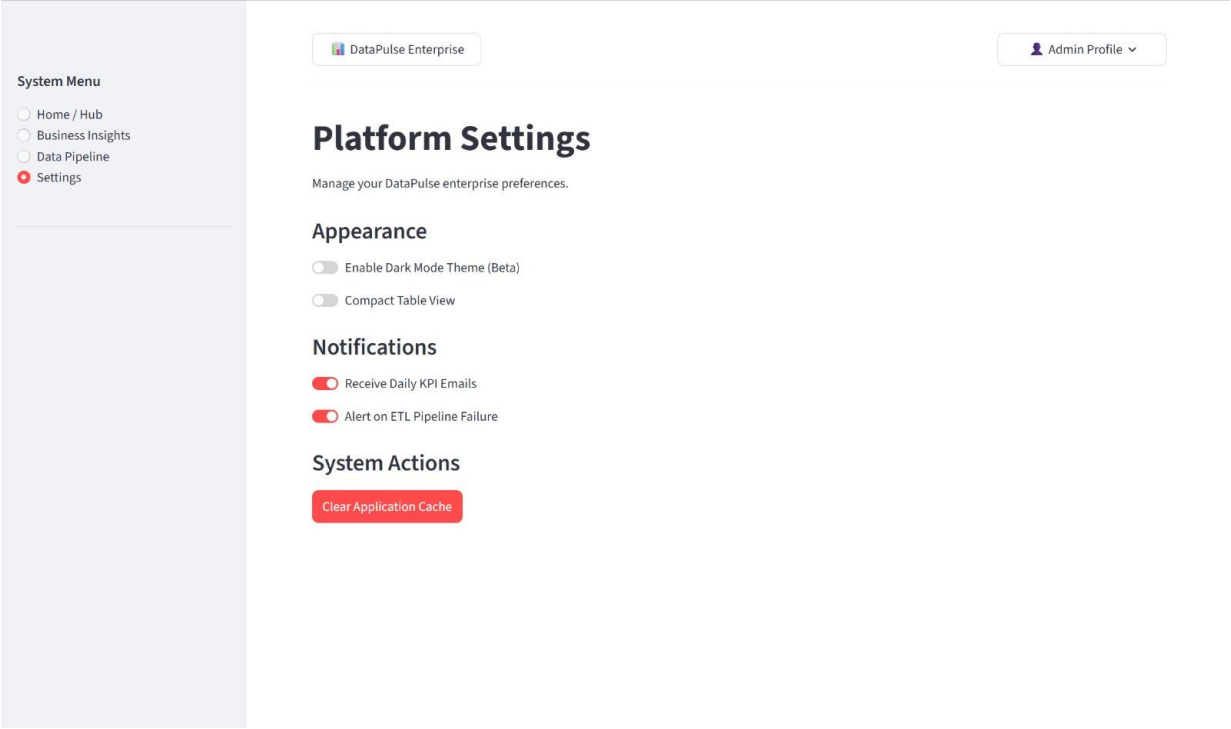
Google Apple

Don't have an account? **Sign up to request access**




System Menu
Home / Hub
Business Insights
Data Pipeline
Settings
DataPulse Enterprise
Admin Profile
Enterprise Hub
Select a module to view real-time data and manage infrastructure.
Core Dashboard
Top-level KPIs, revenue metrics, and global sales trends.
Product Insights
Deep dive into category performance and channel distribution.
ETL Pipeline Monitor
Monitor Supabase sync status, database health, and logs.
Open Dashboard
Open Insights
Open Pipeline
Master Data Viewer
Query and export raw, row-level transactional data.
Prophet ML Engine
Review 30-day AI forecasts and model confidence intervals.
System Settings
Manage API keys, dark mode, and user permissions.
View Data Table
View Predictions
Open Settings


System Menu
Home / Hub
Business Insights
Data Pipeline
Settings
DataPulse Enterprise
Admin Profile
Cloud Infrastructure Status
PostgreSQL Database
Ping Supabase Node
Connection Pool: Active
processed_sales_data: 112,650 rows synced
Prophet ML Engine
AI Model: Offline / Awaiting Data
sales_forecast: Table not found



Chapter 4: Implementation

4.1 Development Environment (Tools and Technologies)

- Programming Language: Python 3.13

- Frontend: Streamlit

- Visualization: Plotly

- Data Processing: Pandas

- Database Connectivity: SQLAlchemy + psycopg2

- Database: Supabase PostgreSQL

- Environment Management: python-dotenv, venv

4.2 System Modules Description

- data_ingestion.py: Reads raw CSVs, cleans headers, and loads SQL tables.

- data_transformation.py: Joins raw tables, engineers features, and publishes processed dataset.

- utils.py: Provides shared database connection utility.

- app.py: Hosts login, navigation, KPI dashboard, forecasting panel, and monitoring screens.

4.3 Frontend Implementation

The frontend is implemented in Streamlit with custom CSS for layout and card styling. Pages include Login, Hub, Business Insights, Data Pipeline, and Settings.

4.4 Backend Implementation

Backend logic is implemented in Python modules responsible for ingestion and transformation workflows, with error handling around database operations and file processing.

4.5 Database Implementation

Supabase PostgreSQL stores raw and processed tables. The processed_sales_data table acts as the main analytics source.

4.6 Integration of System Components

The application integrates ingestion, transformation, and dashboard components through a common database layer and shared utility functions.

Chapter 5: Testing and Evaluation

5.1 Testing Strategy

Testing combines module-level validation for ingestion/transformation scripts with UI-level checks for navigation, filters, and chart rendering.

5.2 Test Cases and Results

- TC-01: Valid database URL returns active engine connection. Result: Pass.

- TC-02: Ingestion reads all CSV files from data/raw. Result: Pass.

- TC-03: Transformation produces non-empty processed table with expected fields. Result: Pass.

- TC-04: Dashboard loads KPIs for valid date ranges. Result: Pass.

- TC-05: Forecast panel gracefully handles missing forecast table. Result: Pass.

5.3 System Validation

System behavior was validated by running end-to-end flow: ingestion, transformation, and dashboard inspection.

5.4 Performance Evaluation

For sprint-scale datasets, dashboard interactions are responsive and batch loading via chunksize supports stable uploads.

5.5 User Feedback

Preliminary internal team feedback indicates that module navigation and KPI summaries are clear and useful for sprint goals.

Chapter 6: Conclusion and Future Work

6.1 Summary of the Project

DataPulse now provides an integrated analytics path from raw eCommerce data to business dashboards with documented system design.

6.2 Achievements

- Implemented ingestion and transformation modules with cloud persistence.

- Built multi-screen Streamlit interface with KPI, charts, and monitoring views.

- Produced UML and architecture artifacts aligned to implemented modules.

- Prepared comprehensive Sprint 2 report structure with chapter-level detail.

6.3 Limitations

- Forecast lifecycle automation is not yet complete.

- Role-based access controls are currently basic.

- Advanced enterprise integrations are scheduled for future sprints.

6.4 Future Enhancements

- Add automated scheduling and alerting for pipeline runs.

- Expand forecasting with model diagnostics and retraining policy.

- Introduce role-based authorization and audit trails.

- Add exportable reporting and deeper drill-down analytics.

References

- Pandas Documentation - https://pandas.pydata.org/docs/

- Streamlit Documentation - https://docs.streamlit.io/

- SQLAlchemy Documentation - https://docs.sqlalchemy.org/

- Plotly Python Documentation - https://plotly.com/python/

- Supabase Documentation - https://supabase.com/docs

Appendices

A. Source Code Snippets

Key snippets are available in app.py, src/utils.py, src/components/data_ingestion.py, and src/components/data_transformation.py.

B. User Manual

1. Run ingestion. 2. Run transformation. 3. Launch app.py in Streamlit. 4. Log in and navigate modules.

C. Screens / UI Designs

UI and screen design assets are included from Iteration 1 and integrated into this report.

D. Additional Diagrams

System architecture and ER schema diagrams are included above.

Required Sections Snapshot

Project title: DataPulse - Enterprise eCommerce Analytics Platform

Problem Statement: Included in Chapter 1.2

Functional Requirements: Included in Chapter 2.4

User stories: Included in Sprint Backlog section

Design (UI Screens): Included using Iteration 1 design assets

Scrum Board: Trello update and screenshots included in submission package

Work division: Included below

Work Division

- Samiullah Shahid (24I-2520) - Group Leader: Led Sprint 2 planning and coordination, maintained Scrum practices, tracked Trello progress, and validated deliverable completeness.

- Rafay Mir Khattak (24I-2603): Led system interface and design consistency tasks, supported UML refinement, and ensured design-to-document alignment across iterations.

- Muhammad Umar Nadeem (24I-2513): Led implementation and integration, developed core application modules, automated diagram/report generation, and consolidated final deliverable artifacts.